ClinicDesk

Appointment desk for a small clinic, from laptop to a monitored EKS cluster

DevOps Final Capstone
Presented by: TODO your name
github.com/RJSIN147/DevopsCapstoneProject

Project overview

What the app does

- Front desk tool for a clinic with a few doctors
- Book a patient into a doctor's slot
- Check in on arrival, complete when they leave, or cancel
- Refuses to double-book a doctor (409 Conflict)
- Pause a doctor so nobody can book them
- Live counts: today, waiting room, completed, cancelled
- Small app on purpose, the project is the delivery pipeline around it

15
REST endpoints

24
pytest tests

2
tables, Alembic

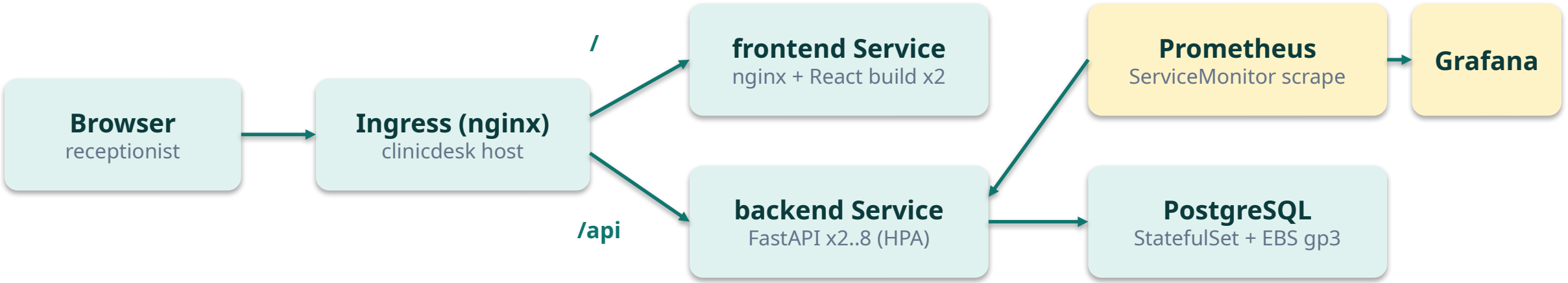
7
CI/CD jobs

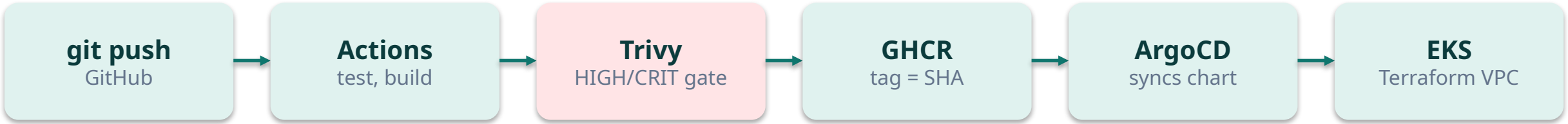
React · FastAPI · PostgreSQL
Docker · GitHub Actions · Trivy · GHCR · Terraform · EKS · Helm · ArgoCD · Prometheus · Grafana

Tech stack

Frontend	React 19, Vite 8, plain responsive CSS, nginx-unprivileged
Backend	Python 3.12, FastAPI, SQLAlchemy 2, Pydantic 2
Database	PostgreSQL 17, Alembic migrations
Testing	pytest + TestClient on in-memory SQLite, node:test, ruff
Containers	Docker multi-stage builds, non-root users, Docker Compose
CI/CD + registry	GitHub Actions, GitHub Container Registry
Security	Trivy image scan gate, Trivy secret scan, SARIF to GitHub
Infrastructure	Terraform, AWS VPC, EKS 1.34, EBS CSI driver
Kubernetes	Helm chart, ingress-nginx, HPA, PDB, ArgoCD
Observability	kube-prometheus-stack: Prometheus, Grafana

Architecture



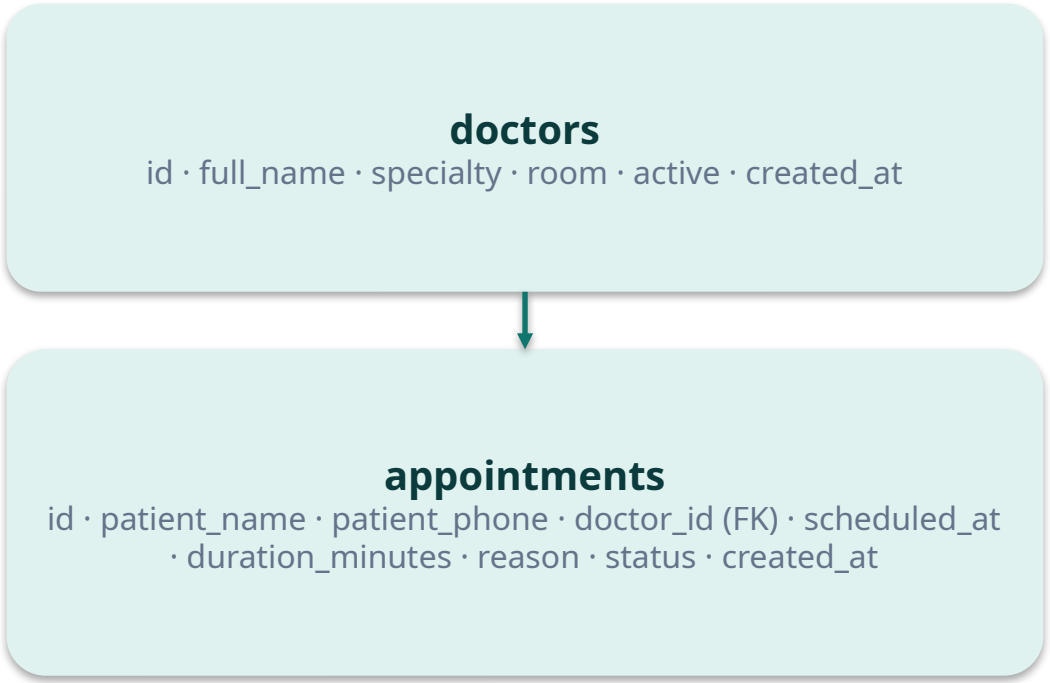


The browser only calls /api on its own origin. nginx (compose) or the Ingress (Kubernetes) routes it to FastAPI, so no internal hostnames leak into the frontend build.

The application

FastAPI + SQLAlchemy + Alembic, React on the front

- Doctors: list, get, add, update/pause, delete
- Appointments: list with filters, get, book, update, delete, stats
- /health for liveness, /ready checks the DB (503 if down)
- /metrics for Prometheus, plus my own counters
- Business rules in the API, not only the UI
  - overlapping bookings for a doctor -> 409
  - paused doctor -> 422, doctor with history can't be deleted
  - all times stored in UTC



Migrations: 0001 creates both tables, 0002 seeds four doctors. Pods take a Postgres advisory lock before migrating.

CI/CD pipeline

GitHub Actions, runs on every push and PR to main

1 Test

- backend: ruff + pytest
- frontend: npm test + build
- helm lint, terraform validate
- Trivy secret scan

2 Build + scan

- docker build backend/frontend
- Trivy full report + SARIF
- gate: fixable HIGH/CRITICAL fails
- push to GHCR as :<sha>

3 Smoke deploy

- kind cluster in the runner
- load the pushed images
- helm upgrade --install
- helm test + book an appointment

4 GitOps

- write SHA into values-prod.yaml
- ArgoCD sees the commit
- rolls EKS to the new image
- (opt-in repo variable)

Tests fail -> nothing is built. Scan fails -> nothing is pushed. Every image is tagged with the commit SHA, so any pod traces back to the exact commit.

DevSecOps

Security checks that run on every commit

- Trivy scans OS packages and Python/npm libs in both images
- Pipeline fails on fixable HIGH or CRITICAL CVEs
- Trivy secret scan over the repo
- Containers: non-root, drop ALL caps, no privilege escalation
- Backend root filesystem is read-only
- No credentials in git: AWS from CLI profile, DB + Grafana passwords from k8s secrets

Real finding from my pipeline

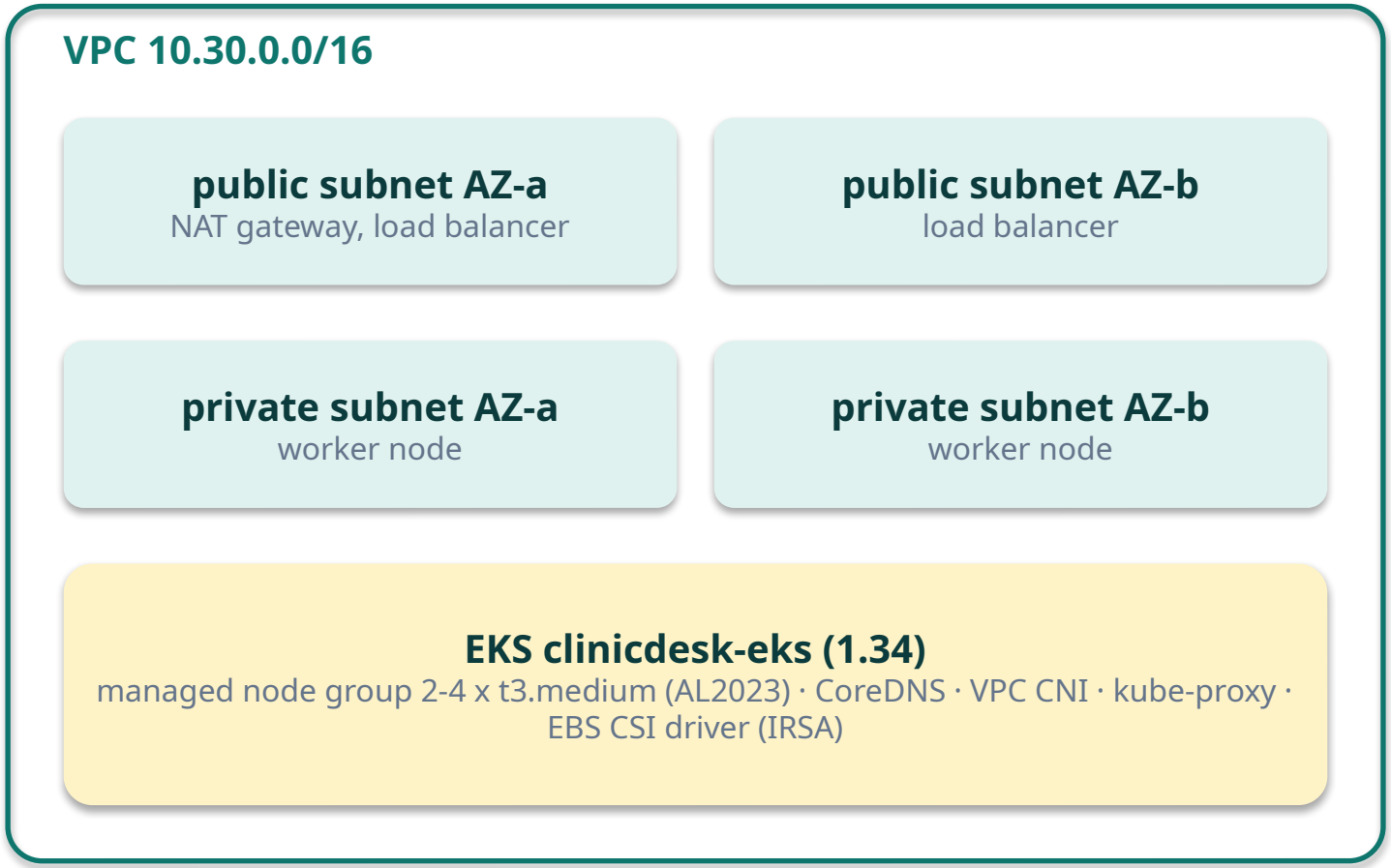
First run failed on 4 HIGH CVEs (urllib3, msgpack, setuptools). They were pip's own vendored libraries inside the image venv. Fix: uninstall pip after installing deps, it isn't needed at runtime.

Latest green run

backend: 0 fixable HIGH/CRITICAL, Python libs clean (166 Debian base findings with no fix yet). frontend: 1 MEDIUM in Alpine.

Infrastructure as Code

Terraform, AWS ap-south-1



- terraform-aws-modules for VPC + EKS
- subnets tagged so load balancers land in the right place
- terraform.tfvars.example, no keys in git
- init -> plan -> apply -> destroy
- gp3 StorageClass applied after, EKS 1.30+ has no default
- Cost: a few dollars a day, destroyed after the demo

Kubernetes + Helm + ArgoCD

One chart, three values files

Deployments
backend (init waits for DB), frontend

StatefulSet
postgres + PVC

Services
ClusterIP for all three

Ingress
/ -> frontend, /api -> backend

HPA
backend 2..8 pods at 60% CPU

PDB
min 1 backend pod in drains

ServiceMonitor
only if the CRD exists

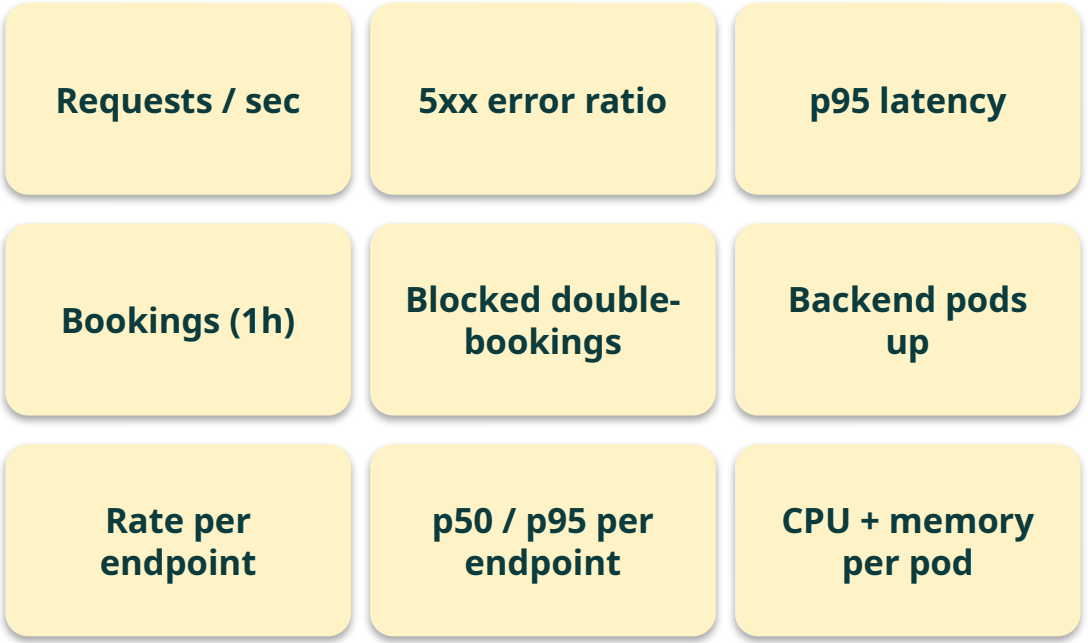
helm test
checks /ready + proxy

- values-dev.yaml: minikube, local images, clinicdesk.local
- values-prod.yaml: EKS, GHCR images by SHA, gp3, secret DB password
- Probes: startup + liveness on /health, readiness on /ready
- Pod Security restricted (warn/audit), ResourceQuota on the namespace
- ArgoCD Application on helm/clinicdesk, auto sync + self heal
- CI deploys the same chart into kind on every push

Observability

kube-prometheus-stack + a dashboard that ships with the chart

- prometheus-fastapi-instrumentator: request count, latency histogram, status codes
- My own counters: bookings, blocked double-bookings, status changes
- ServiceMonitor -> Prometheus scrapes backend every 15s
- Grafana sidecar loads the dashboard ConfigMap automatically
- Load test script to drive the panels and the HPA



TODO: swap in a real Grafana screenshot here

Troubleshooting and key learnings

Things that actually broke

- Init container stuck: pg_isready said "no attempt"
  - uid 10001 has no passwd entry, needed -U <user>
- Two backend pods migrating at once
  - fixed with a Postgres advisory lock in alembic/env.py
- Trivy failing on pip's vendored libraries
  - pip removed from the runtime image
- nginx as non-root can't bind port 80
  - nginx-unprivileged on 8080, Service maps 80 -> 8080

Lab exercises in troubleshooting/

- Wrong image tag -> ImagePullBackOff
- Selector typo -> Service with no endpoints
- Bad DB host -> CrashLoopBackOff

Takeaways

- Readiness != liveness, and it matters for rollouts
- Gates only help if they really stop the pipeline
- Tag by SHA, never latest
- Destroy the cloud stuff when you're done

What I'd do next

- Staff login (OIDC) and an audit log, it's patient data
- Move Postgres to RDS Multi-AZ with backups
- TLS with cert-manager and a real domain
- Swap ingress-nginx (retired upstream) for Gateway API / AWS Load Balancer Controller
- Alerts in Alertmanager: error ratio, p95 latency, pods not ready
- Remote Terraform state in S3 with locking

Thanks! Questions?